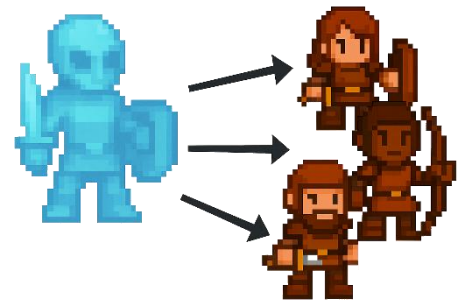


8.3.3 Eigene Klassen implementieren

Bisher hat das Framework sich um unsere Klassen gekümmert, doch das ändern wir jetzt. In den folgenden Kapiteln werden Sie selbst einige Klassen des Frameworks neu implementieren. Wir beginnen dabei mit unserem Helden, für welchen wir nun das folgende, gekürzte Klassendiagramm verwenden.



Held
x : Integer y : Integer richtung : String name : String typ : String weiblich : Boolean
Held (xp,yp : int, richtung : str, w : bool) geh() links() rechts() zurueck()

Aufgabe 1

Laden Sie in der `schueler.py` Level 35. Ein leeres Level erscheint, in dem eigentlich unser Held stehen sollte. Das Framework kümmert sich selbst um die Erzeugung der Objekte anhand der Level-Datei, aber dazu muss die Helden-Klasse auch verfügbar sein.

a) Ergänzen Sie in Ihrem bekannten Programmierbereich die folgenden Zeilen:

```
class Held:
    def __init__(self,xp,yp,richtung,w):
        self.x = 0
```

Führen Sie das Programm aus.

b) Ergänzen Sie nun selbstständig die fehlenden Befehle, um der neuen Helden Klasse alle Attribute zuzuweisen. Verwenden Sie dabei sinnvolle Standardwerte, welche dem Klassendiagramm entsprechen. Den Standardnamen eines Helden dürfen Sie frei wählen, der Typ des Helden ist „Held“.

- c) Möglicherweise ist Ihnen bereits aufgefallen, dass das Level nicht abgeschlossen werden kann, obwohl alle Attribute gesetzt und der Held sichtbar ist. Das liegt daran, dass im Level genau festgelegt ist, wo der Held steht, in welche Richtung er schaut und ob „er“ weiblich ist. Letzteres steuern Sie über `level.lade(35, weiblich=...)` sogar selbst!

Der Konstruktor des Helden erhält über Parameter genau diese Standardwerte:

```
Held(xp,yp : int, rp : str, w : bool)
```

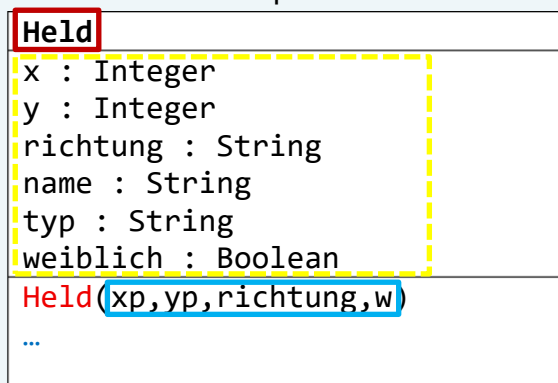
Die Parameter sollten Sie dann auch verwenden. Ändern Sie die Zeilen daher wie folgt ab:
`self.x = xp`

- d) Der Parameter für die Richtung heißt wie das Attribut selbst. Der Parameter für weiblich enthält ebenfalls kein p. Macht dies Probleme?
- e) Sobald Ihre Lösung **Level 35** löst („Level abgeschlossen“), laden Sie auch **Level 36** mit der gleichen Lösung. Ist dies ebenfalls direkt abgeschlossen, so haben Sie das Level gelöst und können fortfahren!

Merke:

Eine Klasse kann anhand des Klassendiagramms schnell implementiert werden. Eingeleitet wird eine Klassendefinition mit dem Schlüsselwort `class`, gefolgt vom Namen der Klasse und einem Doppelpunkt.

Beispiel



```
6 class Held:
7     def __init__(self, xp, yp, richtung, w):
8         self.x = xp
9         self.y = yp
10        self.richtung = richtung
11        self.name = "Namenloser Held"
12        self.typ = "Held"
13        self.weiblich = w
```

Alle Methoden der Klasse werden dann eingerückt implementiert, beginnend beim **Konstruktor** der Klasse. Obwohl er beim Aufruf wie die Klasse selbst heißt, heißt er bei der Implementierung `__init__` und enthält als Parameter immer `self`. Die übrigen Parameter werden aus dem dem Klassendiagramm übernommen.

Alle Attribute werden dann in der Form `self.attribut` initialisiert. Entweder:

- Durch Verwendung eines Parameters, z.B. bei `x` oder `weiblich`
- Durch Setzen eines sinnvollen Standardwerts, z.B. bei `typ` oder `name`.

Aufgabe 2

Im Konstruktor können nicht nur stumpfe Werte oder Parameter gesetzt werden. Ändern Sie den Konstruktor so um, dass „Namenloser Held“ bei männlichen und „Namenlose Heldin“ bei weiblichen Helden als Name gesetzt wird.

Hinweis: Obwohl es sinnvoll wäre, tun Sie dies nicht beim Attribut Typ, da in diesem Fall die Spiellogik des Frameworks gebrochen wird.

Aufgabe 3

Jetzt fügen wir der Helden-Klasse die Fähigkeit hinzu, sich zu bewegen. Unser Ziel wird es sein, die Funktionalität des Frameworks Schritt für Schritt wiederherzustellen, so dass Sie nach Implementierung der Klassen(methoden) **die Level mit der Tastatur** lösen können. Sie müssen also ab sofort keine Programme zum Lösen der Level selbst mehr schreiben.

- a) Öffnen Sie **Level 37**. Dieses Level erfordert zum Lösen, dass sich der Held zum markierten Zielfeld bewegt und alle Bewegungsmethoden implementiert sind.
- b) Fügen Sie der Klasse Held eine neue Methode **geh** hinzu, welche die gleiche Einrückung wie der Konstruktor besitzt:

```
def geh(self):  
    if self.richtung == "up":  
        self.y = self.y + ...
```

Implementieren Sie die Methode **geh** vollständig. Testen Sie Ihre Implementierung, indem Sie Level 37 starten und den Helden mit Pfeiltaste oben  bewegen.

- c) Obwohl die Methode **geh** laut Klassendiagramm keine Parameter besitzt, müssen Methoden einer Klasse in Python immer den Parameter **self** erhalten. Testen Sie jetzt den Befehl `held.geh()` im Hauptprogramm, also dort, wo Sie auch zuvor programmiert hatten (Einrückung 0, z.B. direkt vor der Zeile `framework.starten()`).
- d) Implementieren Sie auf die gleiche Art und Weise die Methoden `links`, `rechts` und `zurueck`, um Level 37 mithilfe der Pfeiltasten zu lösen.

Merke:

Methoden einer Klasse können nach dem Konstruktor mit `def methode(...)` auf gleicher Einrückungsebene hinzugefügt werden.

Innerhalb der Klasse können die Attribute mit `self.attribut` referenziert werden. Das `self` ist dabei sozusagen eine **Selbstreferenz**, auf genau das Objekt, welches gerade die Methode ausführt. Dieses `self` muss als Parameter jeder Klassenmethode aufgeführt werden, obwohl dieser beim Aufruf später nicht mehr sichtbar ist.

Beispiel

Held
x : Integer
y : Integer
richtung : String
name : String
typ : String
weiblich : Boolean
<code>Held(xp,yp,richtung,w)</code>
<code>geh()</code>
<code>links()</code>
...

```
6 class Held:
7     def __init__(self, xp, yp, richtung, w):
8         self.x = xp
9         ...
10        self.weiblich = w
11
12    def geh(self):
13        if self.richtung == "down":
14            self.y = self.y + 1
15        elif self.richtung == "right":
16            ...
17
18 # Hauptprogramm-Ebene
```

Der erste Befehl, welcher sich wieder auf der Einrückungsstufe der Klassendefinition befindet, ist nicht mehr Teil der Klasse, sondern z.B. Teil des Hauptprogramms. Klassen können theoretisch auch auf Ebene eines Unterprogramms definiert werden.

Übungsaufgaben

- Passen Sie den Konstruktor so an, dass nur `up`, `down`, `left` und `right` als Werte für `richtung` akzeptiert werden. Ansonsten wird `down` als Standard gesetzt
- Passen Sie den Konstruktor so an, dass nur positive Werte für `x` und `y` gesetzt werden können
- Schreiben Sie einen kleinen Namensgenerator, welcher beim Erzeugen des Helden einen zufälligen Namen erzeugt und setzt (entweder als Unterprogramm `erzeuge_name()` oder direkt im Konstruktor